

Guía de Laboratorio N°11

Automatización de Archivos con Python

Docente: Camilo Alejandro Fuentes Beals - PhD
Programación Científica

1. Ejercicios

Instrucciones Generales

1. Trabaja individualmente en un Jupyter Notebook (.ipynb).
2. Todos los bloques de código deben ejecutarse sin errores antes de entregar.
3. Comenta cada sección con texto Markdown que explique tu razonamiento.
4. El dataset requerido es: datos_11.zip (En Canvas).
5. Tiempo estimado: 90 minutos. Duración del bloque de laboratorio: 2 horas.
6. Entrega: repositorio GitHub personal, rama main, antes del término de la sesión.

1. Introducción y Contexto Pedagógico

Eres analista de datos en TechDistribucion S.A., una empresa que distribuye equipos tecnológicos en Chile. El área comercial te entregó una carpeta con 12 archivos CSV (uno por mes) con los registros de ventas de 2024, un catálogo de productos en JSON y un resumen ejecutivo en Excel con dos hojas. Tu tarea es consolidar, analizar y exportar un reporte ejecutivo completo usando Python y Pandas.

1. Desarrollo de Ejercicios

Completa los 5 ejercicios a continuación en tu Jupyter Notebook. En cada ejercicio incluye el enunciado, el código de apoyo y las preguntas de reflexión.

Ejercicio 1: Inspección inicial de archivos

20 Puntos

Antes de procesar cualquier dataset, es fundamental inspeccionarlo. En este ejercicio cargaras uno de los archivos CSV mensuales y analizaras su contenido.

1. Lee el archivo `ventas_enero.csv` usando `pd.read_csv()` con `parse_dates=["fecha"]`. Muestra las primeras 5 filas con `.head()`.
2. Ejecuta las siguientes inspecciones sobre el DataFrame y registra tus hallazgos en la tabla de abajo:

```
df.shape          # dimensiones del DataFrame
df.dtypes         # tipo de dato por columna
df.isnull().sum() # valores faltantes por columna
df.describe()     # estadísticas descriptivas
```

Completa la siguiente tabla con los resultados de tu inspección:

Columna	Tipo de dato	Valores faltantes	Observación
fecha	<code>datetime64[ns]</code>	0	Verificar que el rango de fechas sea consistente
producto	<code>object</code>	0	Tipo object (string). Revisar si hay inconsistencias en nombres
vendedor	<code>object</code>	1	1 valor faltante. Decidir estrategia: eliminar la fila, imputar con "Desconocido" o inferir según contexto.
region	<code>object</code>	0	verificar que los valores correspondan a un catálogo fijo de regiones para evitar duplicidad
cantidad	<code>int64</code>	0	Tipo int64. Validar que no existan valores negativos o ceros que no tengan sentido de negocio.
precio_unitario	<code>int64</code>	0	Tipo int64. para solo considerar valores enteros
total	<code>int64</code>	0	

Basándote en la inspección: escribe un código Python que filtre los registros donde vendedor no es nulo (`notna()`) y muestra cuantas filas quedan.

Ejercicio 2: Consolidacion por lotes con glob y pd.concat

30 puntos

Contexto

El patrón glob -> loop -> concat es el núcleo de la automatización de archivos. En este ejercicio lo implementarás completo sobre los 12 CSV de ventas.

1. Usa Path y `.glob('*.csv')` para listar todos los archivos de la carpeta `datos/ventas/`. Imprime cuantos archivos encontraste.
2. Implementa el pipeline completo usando la siguiente estructura como punto de partida:

```
from pathlib import Path
import pandas as pd

carpeta = Path('datos/ventas')
archivos = sorted(carpeta.glob('*.csv'))

dfs = []
```

```

for archivo in archivos:
    df = pd.read_csv(archivo, parse_dates=['fecha'])
    df['mes'] = _____ # extrae el nombre del mes del archivo
    dfs.append(df)

consolidado = pd.concat(dfs, ignore_index=_____)
  
```

- Valida la consolidación: verifica que el total de filas del DataFrame consolidado es igual a la suma de las filas de cada archivo individual.

Pista:

Para extraer el nombre del mes desde el nombre del archivo, usa `archivo.stem.replace('ventas_', '')`. El método `.stem` devuelve el nombre del archivo sin la extensión.

- Registra los resultados de tu validación:

Verificación	Resultado
Archivos CSV encontrados (glob)	12
Filas en el DataFrame consolidado	397
Suma de filas individuales	30+31+62+31+29+31+30+31+31+30+31+30
Validación exitosa (Si / No)	si
Columnas del DataFrame final	8

Ejercicio 3: Análisis del consolidado y exportación

30 puntos

Con el DataFrame consolidado del ejercicio anterior, realizaras un análisis de ventas y exportaras los resultados en dos formatos.

- Calcula las siguientes métricas usando `.groupby()` y registra los resultados:

Métrica	Código Python	Resultado
Total, de ventas CLP por mes	for grupo in consolidado.groupby("mes")["total"]: print(grupo[1].sum())	17987000
Producto más vendido (cantidad)	for grupo1 in consolidado.groupby("producto")["cantidad"]: print(grupo1[1].sum().max())	WEBCAM =176
Mes con mayor ingreso	for grupo2 in consolidado.groupby("mes")["total"]: print(grupo2[1].sum().max())	Diciembre= 28508000
Promedio de venta por transaccion	consolidado["total"].mean()	543584.3828715365

- Exporta el DataFrame consolidado a un archivo `ventas_2024_consolidado.csv` en la carpeta `datos/salida/`. Usa `index=False` y `encoding='utf-8'`.
- Exporta el mismo DataFrame a un archivo Excel `reporte_anual_2024.xlsx` con tres hojas usando `pd.ExcelWriter`:

```

with pd.ExcelWriter('datos/salida/reporte_anual_2024.xlsx', engine='openpyxl') as writer:
    consolidado.to_excel(writer, sheet_name='Datos_Completos', index=False)
    resumen_mensual.to_excel(writer, sheet_name='Resumen_Mensual', index=False)
  
```

```
resumen_productos.to_excel(writer, sheet_name='Resumen_Productos', index=False)
```

Verifica que el Excel se generó correctamente leyendo sus hojas con `pd.ExcelFile` y reporta sus nombres:

```
with pd.ExcelFile('datos/salida/reporte_anual_2024.xlsx') as libro:  
    print(libro.sheet_names)
```

Ejercicio 4: Manejo de errores y casos borde

20 puntos

Saber anticipar y resolver errores es tan importante como escribir código correcto. En este ejercicio identificaras y corregirás cuatro situaciones problemáticas.

Para cada caso: (a) ejecuta el código con error y registra el mensaje que produce, (b) aplica la corrección indicada y (c) confirma que el código corregido funciona sin errores.

Caso 1 — `FileNotFoundException`:

```
# Código con error  
pd.read_csv('datos/ventas/ventas_enero.CSV') # extension en mayusculas  
  
# Correccion: usa .exists() para verificar antes de leer  
ruta = Path('_____')  
if ruta.exists():  
    df = pd.read_csv(ruta)  
else:  
    print('Archivo no encontrado:', ruta.resolve())
```

Mensaje de error observado: Archivo no encontrado: C:\Users\jcche\OneDrive\Desktop\lab11_____

Caso 2 — `EmptyDataError` (glob sin resultados):

```
# Código con error  
carpeta_erronea = Path('datos/ventas_2025')  
dfs = [pd.read_csv(a) for a in carpeta_erronea.glob('*.csv')]  
pd.concat(dfs) # ValueError si dfs esta vacio  
  
# Correccion: verifica antes de concatenar  
archivos = list(carpetas_erroneas.glob('*.csv'))  
if len(archivos) == _____:  
    print('No se encontraron archivos en:', carpeta_erronea)  
else:  
    resultado = pd.concat([pd.read_csv(a) for a in archivos], ignore_index=True)
```

Mensaje de error observado: el error fue que estaba leyendo una carpeta que no existe

Caso 3 — `UnicodeDecodeError`:

```
# Código con error (el archivo usa encoding cp1252)  
pd.read_csv('datos/archivo_cp1252.csv') # falla sin encoding correcto  
  
# Correccion: especifica el encoding  
pd.read_csv('datos/archivo_cp1252.csv', encoding='_____')
```

Mensaje de error observado: utf-8' codec can't decode byte 0xe9 in position 17: invalid continuation byte

Caso 4 — Asignación de columna fuera del loop:

```
# Código incorrecto: la columna 'mes' se asigna fuera del loop
dfs = []
for archivo in sorted(Path('datos/ventas').glob('*.csv')):
    df = pd.read_csv(archivo)
    dfs.append(df)
df['mes'] = archivo.stem # ERROR: solo modifica el ultimo df

# Escribe el código corregido aquí:
```

5. Criterios de Evaluación

La guía se evalúa sobre 100 puntos según la siguiente rúbrica. La nota final se convierte a la escala 1.0 – 7.0 con nota mínima de aprobación 4.0 (≥ 60 puntos).

Criterio	Logrado (100%)	En Desarrollo (60%)	Insuficiente (0%)
Ejercicio 1: Inspección inicial (20 pts)	Lee correctamente el CSV con <code>parse_dates</code> , completa la tabla de inspección sin errores y filtra registros con <code>notna()</code> .	Lee el CSV pero la tabla de inspección tiene valores incorrectos o incompletos, o el filtro no usa <code>notna()</code> .	No lee el CSV correctamente o no entrega la tabla de inspección.
Ejercicio 2: Consolidación (30 pts)	Implementa el pipeline <code>glob-loop-concat</code> completo, agrega columna 'mes' dentro del loop y valida que filas totales coinciden.	El pipeline funciona pero la columna 'mes' se agrega incorrectamente o la validación no es completa.	El pipeline no genera el DataFrame consolidado correctamente.
Ejercicio 3: Análisis y exportación (30 pts)	Calcula las 4 métricas correctamente, exporta CSV y Excel con las hojas requeridas y verifica con <code>ExcelFile</code> .	Calcula al menos 2 métricas, exporta al menos un formato correcto (CSV o Excel).	No exporta ningún archivo o los cálculos presentan errores significativos.
Ejercicio 4: Manejo de errores (20 pts)	Identifica el error en los 4 casos, registra el mensaje correcto y el código corregido funciona sin excepciones.	Identifica y corrige al menos 2 casos correctamente.	Identifica menos de 2 casos o los códigos corregidos producen errores.

⚠ Importante: El Notebook debe ejecutar completamente sin errores para acceder a todo el puntaje. Notebooks con errores de ejecución no corregidos recibirán puntaje máximo de 50 puntos.

3. Formato y Condiciones de Entrega

- **Nombre del archivo:** Apellido_Nombre_Lab11.ipynb
- **Plataforma:** Canvas.
- **Fecha Límite de entrega:** Termino del Laboratorio.
- **Formato:** Jupyter Notebook (.ipynb).
 - a. No se aceptan archivos .py ni .html.
 - b. El Código tiene que generar todos los archivos pedidos.
- **Ejecución:** El Notebook debe estar ejecutado (Kernel → Restart & Run All) antes de subir.
- **Celda inicial:** Incluir nombre completo, RUT y fecha en la primera celda Markdown.